



Casita de Romila

Sistema de Gestión para Panadería y Cafetería

 **MANUAL TÉCNICO**

Versión de la aplicación: 1.0

Versión del manual: 1.0

Fecha: Septiembre de 2026

Audiencia: desarrolladores, administradores y personal de soporte

Índice de contenido

1. Resumen del proyecto
2. Arquitectura y stack tecnológico
3. Estructura de directorios
4. Requisitos del entorno
5. Instalación
6. Configuración
7. Modelo de datos
8. Roles y permisos
9. Autenticación y sesiones
10. Lógica de negocio clave
11. Rutas de la aplicación
12. Interfaz de usuario
13. Pruebas y calidad
14. Despliegue y producción
15. Mantenimiento, respaldos y solución de problemas
16. Registro de cambios

1. Resumen del proyecto

Casita de Romila es un sistema web de gestión para una panadería y cafetería. Está pensado para uso local en un solo establecimiento, operado desde el puesto de caja y el teléfono o PC del dueño.

Funcionalidades principales:

- **Punto de venta (POS)** con múltiples métodos de pago (efectivo, Biopago, pago móvil, PDV, crédito) y precios recalculados desde el servidor (nunca desde el cliente).
- **Comandas:** pedidos a cocina con entrega por unidades que se convierten en venta real al cobrar.
- **Inventario**, movimientos (compras/ajustes/devoluciones), **producción** por lotes, **mermas** y **consumo interno** (multilínea, con costo en USD).
- **Créditos a clientes** con saldo en USD, límites y cobro posterior (ingreso registrado en el día del cobro).
- **Tasa de cambio** diaria con historial y **tasas congeladas** por venta.
- **Reportes:** ventas, mermas vs consumo, producción vs ventas, margen, top días, horarios, etc., con exportación CSV.
- **Usuarios y roles** con permisos por módulo.

El sistema fue desarrollado sobre imágenes de demostración para este manual. Las cifras de pruebas citadas corresponden a la versión documentada.

2. Arquitectura y stack tecnológico

2.1 Vista general

Aplicación monolítica **Laravel** con vistas *server-side* (Blade) y comportamiento interactivo ligero con **Alpine.js**. El frontend se compila con **Vite**. Los datos viven en **PostgreSQL** en producción; las pruebas usan SQLite en memoria.

Capa	Tecnología	Detalle
Lenguaje / Framework	PHP 8.3 · Laravel 13	Composer: <code>laravel/framework ^13.17</code> . Se usa el arranque clásico (bootstrap/app.php), sin módulos.
Base de datos (prod)	PostgreSQL	Conexión <code>pgsql</code> . Numeración serializada con <code>pg_advisory_xact_lock</code> .
Base de datos (tests)	SQLite en memoria	Configurada en <code>phpunit.xml</code> ; sin servicio externo.
Frontend	Bootstrap 5 · Alpine.js · Blade	Bootstrap para estructura, Alpine para interactividad en POS/carritos/modales, Blade + <code>@vite</code> .
Gráficos	Chart.js	Gráficos de barras en dashboard y reportes.
Estilos	CSS propio + oklch	Tokens de diseño en <code>resources/css/app.css</code> (paleta rosa/menta), fuentes offline vía <code>@fontsource</code> (Insaniburga + Montserrat).
Autenticación	Login por username	Sin email; cuentas con <code>users.username</code> único. Rate limit y control de usuarios activos a mitad de sesión.
Ejecución local	<code>php artisan serve</code>	El sistema está pensado para correr en un PC del local; el "lanzador" (<code>launcher/romila-launcher.ps1</code>) levanta el servidor y limpia sesiones.

2.2 Decisiones de diseño

- **Precios autoritativos del servidor:** `CheckoutService::execute()` recalcula todo desde la BD ignorando precios/nombres enviados por el cliente.
- **Promedios monetarios en doble moneda:** el costo/precio/stock se almacenan en **USD**; el valor en Bs se deriva con la tasa del día. Se soporta *redondeo* a múltiplos (5/10/...) vía la columna `round_bs`.
- **Permisos por módulo (gates dinámicos):** definidos en `AppServiceProvider::boot()` desde el catálogo de `Permission::CATALOG`.
- **Transacciones para dinero:** checkout, cobro de crédito y anulación de ventas usan `DB::transaction` con `lockForUpdate()` y re-verificación.

3. Estructura de directorios

```
casita-de-romila/
├─ app/
│   ├─ Enums/
│   │   ├─ ProductType.php      # Inventariable | Produccion | Demanda
│   │   └─ Schedule.php        # Horarios (manana/ambos/noche...)
│   └─ Exceptions/
│       └─ CheckoutException.php # Errores de negocio del checkout
├─ Http/
│   ├─ Controllers/            # 18 controladores (ver §11)
│   │   └─ Auth/LoginController.php
│   └─ Middleware/
│       ├─ EnsureUserIsActive.php
│       └─ RegistrarLanzador.php
├─ Models/                    # 20 modelos Eloquent (ver §7)
├─ Providers/
│   └─ AppServiceProvider.php  # Gates desde Permission::CATALOG
├─ Services/
│   └─ CheckoutService.php     # Fuente de verdad del checkout
├─ Support/
│   ├─ Dates.php              # Rango por defecto (inicio de mes)
│   ├─ Like.php               # Filtro LIKE portable (sqlite/pgsql)
│   └─ Pricing.php            # bs(): USD→Bs con redondeo
├─ bootstrap/app.php          # Middleware global y por grupo
├─ config/, database/         # migraciones y seeders
├─ migrations/                # 25 migraciones (ver §7)
├─ seeders/
│   ├─ DatabaseSeeder.php      # Roles+permisos+usuario admin
│   └─ RolePermissionSeeder.php # Idempotente (firstOrCreate)
├─ launcher/
│   └─ romila-launcher.ps1     # Lanzador local (servir + limpiar)
├─ resources/
│   ├─ css/app.css             # Tokens oklch, paleta
│   ├─ js/app.js, js/pos.js    # Bootstrap+Alpine+posApp
│   └─ views/                  # 41 vistas Blade + layouts
├─ routes/web.php              # Todas las rutas (ver §11)
└─ tests/                     # 23 archivos; suite verde 304/990
```

4. Requisitos del entorno

Componente	Versión usada	Notas
PHP	8.3.32 (CLI)	Extensiones: pdo_pgsql, pdo_sqlite, mbstring, openssl, fileinfo, zip.
Composer	2.x	Gestión de dependencias PHP.
Node.js / npm	v24.18.0 / 11.16.0	Compilación de assets con Vite.
PostgreSQL	18	Servicio <code>postgresql-x64-18</code> , escuchando en <code>127.0.0.1:5432</code> . Utilidades <code>psql</code> / <code>createdb</code> en <code>C:\Program Files\PostgreSQL\18\bin</code> .
Navegador	Edge / Chrome / equivalente	Requisito del runtime p. ej. para <code>chromium.launch</code> al generar PDFs/capturas.

5. Instalación

5.1 Clonar e instalar dependencias

```
git clone <repositorio> casita-de-romila
cd casita-de-romila
composer install
npm install
```

5.2 Configurar entorno

```
copy .env.example .env
# editar .env (ver §6)
```

5.3 Base de datos

```
# crear la base desde psql (password interactivo o PGPASSWORD)
psql -h 127.0.0.1 -U postgres -c "CREATE DATABASE casita;"

# migrar y sembrar
php artisan migrate --force
php artisan db:seed --force      # roles, permisos y usuario admin
```

5.4 Compilar assets y arrancar

```
npm run build                # producción
# ó npm run dev              # desarrollo (Vite dev server)

php artisan serve --host=127.0.0.1 --port=8000
```

Acceso inicial: usuario `admin`, contraseña `password` (cambiar en producción).

El `DatabaseSeeder` crea roles/permisos/admin pero **no** crea datos de demostración. En AGENTS.md y el README se menciona un `DemoSeeder` (`php artisan db:seed --class=DemoSeeder`) que **no existe** actualmente; véase §16 para esta corrección de documentación.

5.5 Entorno de desarrollo notable

- Las sesiones usan el driver **database** (tabla `sessions`).
- El “lanzador” `launcher/romila-launcher.ps1` invoca `php artisan sessions:purge` (borra sesiones expiradas y registros huérfanos de la tabla `lanzador_sesiones`) antes de levantar el servidor.

6. Configuración

6.1 Variables principales de `.env`

Variable	Ejemplo	Descripción
<code>APP_URL</code>	<code>http://localhost:8000</code>	URL pública; afecta URLs generadas.
<code>DB_CONNECTION</code>	<code>pgsql</code>	Motor de producción. Pruebas usan <code>sqlite</code> in-memory.
<code>DB_HOST</code> / <code>DB_PORT</code>	<code>127.0.0.1:5432</code>	Servidor PostgreSQL.
<code>DB_DATABASE</code> / <code>DB_USERNAME</code> / <code>DB_PASSWORD</code>	<code>casita</code> / <code>postgres</code> / ...	Credenciales de la base.
<code>SESSION_DRIVER</code>	<code>database</code>	Sesiones persistentes en la tabla <code>sessions</code> .

6.2 Consideraciones

- El `.env.example` trae `DB_PASSWORD=postgres`; el entorno real usa el password del servidor PostgreSQL instalado.
- Para ejecutar con una BD alterna (p. ej. la de demostración) se puede sobrescribir en la misma invocación:
`$env:DB_DATABASE='casita_docs'; php artisan serve ...`.
- Siempre** limpiar vistas compiladas tras tocar templates: `Remove-Item storage/framework/views/*.php -Force` (o `php artisan view:clear`).

7. Modelo de datos

7.1 Inventario de tablas (25 migraciones)

Tabla	Propósito y columnas clave
users	Usuarios: <code>name</code> , <code>username</code> (único, minúsculas), <code>password</code> , <code>is_active</code> , <code>last_login_at</code> , <code>role_id</code> FK.
roles / permissions / permission_role	RBAC: roles (<code>name</code> , <code>slug</code> único, <code>is_system</code>), catálogo de permisos (<code>key</code> , <code>label</code> , <code>module</code>), pivote.
categories	Catálogo de categorías (<code>name</code>).
products	<code>category_id</code> , <code>name</code> , <code>control_type</code> (inventariable/produccion/demanda), <code>cost_price</code> / <code>sale_price</code> (USD), <code>margin_percent</code> , <code>price_override</code> , <code>round_bs</code> , <code>stock_min</code> , <code>stock_current</code> , <code>schedule</code> , <code>is_active</code> , <code>image</code> , <code>notes</code> .
combos / combo_product	Combos con precio propio (<code>sale_price</code> , <code>round_bs</code> , <code>image</code> , <code>is_active</code>). Pivote con <code>quantity</code> (PK compuesta), modelo <code>ComboProduct</code> .
customers	Clientes: <code>name</code> , <code>phone</code> , <code>balance</code> (USD; negativo = deuda), <code>credit_limit_type</code> (libre/monto), <code>credit_limit_amount</code> , <code>is_active</code> .
sales	<code>sale_number</code> (único global), <code>user_id</code> , <code>customer_id</code> , <code>total</code> (Bs), <code>payment_method</code> (incluye <code>credito</code>), <code>amount_paid</code> , <code>status</code> (completada/pendiente/anulada), <code>rate</code> (tasa congelada), <code>paid_at</code> , campos de anulación (<code>cancel_reason</code> , <code>canceled_by</code> , <code>canceled_at</code>).
sale_items	Snapshot: <code>sale_id</code> , <code>product_id</code> / <code>combo_id</code> (nullable), <code>product_name</code> , <code>quantity</code> , <code>unit_price</code> , <code>subtotal</code> .
sale_payments	Pagos: <code>sale_id</code> , <code>method</code> (efectivo/biopago/pago_movil/pdv/transferencia/ credito), <code>amount</code> . El método credito se añadió sin enum nativo (ver §7.2).
credit_movements	Movimientos de crédito: <code>customer_id</code> , <code>sale_id</code> (nullable), <code>user_id</code> , <code>type</code> (cargo/abono/pago), <code>amount</code> (USD), <code>rate</code> , <code>notes</code> .
productions	Lotes: <code>product_id</code> , <code>user_id</code> , <code>quantity</code> , <code>notes</code> .
inventory_adjustments	Movimientos: <code>product_id</code> , <code>production_id</code> / <code>user_id</code> (nullable), <code>type</code> (entrada/salida), <code>quantity</code> , <code>reason</code> (compra/merma/ajuste/venta/devolucion/produccion), <code>notes</code> .
mermas	<code>product_id</code> , <code>user_id</code> , <code>quantity</code> , <code>reason</code> (vencido/danado/otro), <code>notes</code> , <code>type</code> (merma/consumo), <code>cost</code> (USD, nullable). Scopes: <code>mermaType()</code> , <code>consumption()</code> , <code>inRange()</code> .
exchange_rates	<code>rate</code> (decimal 10,2), <code>source</code> (bcv/paralelo/binance/enzona/manual), <code>user_id</code> .
comandas	<code>comanda_number</code> (correlativo diario), <code>user_id</code> , <code>status</code> (montada/entregada/cobrada), <code>customer_name</code> , <code>sale_id</code> (nullable), <code>total</code> .
comanda_items	<code>comanda_id</code> , <code>product_id</code> / <code>combo_id</code> , <code>order_type</code> (local/para_llevar/delivery), <code>note</code> , <code>quantity</code> , <code>unit_price</code> , <code>subtotal</code> , <code>delivered_quantity</code> , <code>delivered_at</code> , <code>collected</code> .
comanda_payments	Pagos previos: <code>comanda_id</code> , <code>amount</code> , <code>method</code> , <code>customer_id</code> , <code>user_id</code> . Solo los <i>delivery</i> se pueden cobrar antes de entregar.
lanzador_sesiones	Registros del lanzador: guarda <code>token</code> como SHA-256 (nunca en claro), linked a sesiones activas.
infraestructura Laravel	<code>sessions</code> , <code>cache</code> , <code>jobs</code> , <code>migrations</code> , <code>password_reset_tokens</code> (templates base).

7.2 Detalles importantes del esquema

- **Pago “credito”:** PHP no crea un enum nativo en PostgreSQL para `sale_payments.method`; la migración de Laravel crea un `varchar(255) CHECK`. La migración `2026_09_12_000001` elimina esa constraint (buscándola en `pg_constraint`) y en SQLite hace `string()->change()`; además hace backfill de movimientos `pago` como métodos reales.
- **Tasa congelada:** `sales.rate` captura la tasa del día de la venta; se rellena con la última tasa $\leq$ a la fecha de creación. `paid_at` es `null` para créditos sin cobrar.
- **Combos:** el pivote usa PK compuesta (`combo_id`, `product_id`) con `quantity`; el modelo es `ComboProduct` (vía `->using()`).

8. Roles y permisos

8.1 Catálogo (8 permisos)

Permiso ((key))	Módulo
<code>manage-users</code>	Usuarios y roles (índice + escritura).
<code>manage-products</code>	Productos y categorías.
<code>manage-inventory</code>	Inventario y producción.
<code>manage-waste</code>	Mermas.
<code>void-sales</code>	Anulación de ventas.
<code>manage-credits</code>	Créditos y clientes.
<code>manage-exchange-rates</code>	Tasa de cambio.
<code>view-reports</code>	Reportes (todas las vistas + CSV).

Abiertos a todo usuario autenticado y activo: dashboard, POS, ventas (consulta) y comandas.

- Roles por defecto: **gerente** (`is_system`, todos los permisos, no eliminable) y **repcionista**.
- Gates registrados dinámicamente en `AppServiceProvider::boot()`; se consultan con `@can` en las vistas y en `routes/web.php`.
- No se puede borrar un rol con usuarios asignados. Los tests reciben permisos mediante el rol asignado por la factory del usuario.

9. Autenticación y sesiones

- **Login por username:** `LoginController::login` (mutator `User::setUsernameAttribute` normaliza a minúsculas).
- **Throttle:** 5 intentos/min por `usuario|IP` (RateLimiter), con mensaje y contador en segundos.
- **Usuarios inactivos:** rechazo en login y expulsión a mitad de sesión vía middleware `EnsureUserIsActive` (grupo web, tras `RegistrarLanzador`).
- **Lanzador:** `RegistrarLanzador` asocia la sesión con un token guardado como SHA-256; `POST /lanzador/cerrar-sesion` cierra sesión sin CSRF (llamado desde PowerShell al cerrar la ventana) con throttle `10,1`.
- **Remember me:** se usan cookies cifradas; en tests hay que reenviar cookies explícitamente (ver `tests/Feature/AuthTest.php`).
- **Driver de sesión:** `database`. El guard de auth es singleton del contenedor: entre requests de un mismo test cachea al usuario, algo a tener en cuenta al simular "sesión perdida".

10. Lógica de negocio clave

10.1 Checkout (CheckoutService)

`CheckoutService::execute(cart, paymentMethod, customerId, userId, comandaId?)` es la única vía de crear ventas (POS y cobro de comandas). Comportamiento:

- Recalcula **precio y nombre desde BD** (`combo_id` → componentes). Se ignora cualquier valor enviado por el cliente (`pos.js` solo manda `product_id` + `quantity`).
- Requiere **tasa vigente**: `currentRate()` lanza `CheckoutException` si no hay `ExchangeRate` registrada.
- Valida **stock** (productos `inventariable` por línea) con `lockForUpdate()`.
- Genera `Sale` + `SaleItems`, descuenta stock, crea `SalePayment` (o `CreditMovement` si es crédito), asigna `sale_number`, vincula la comanda (estado → `cobrada`).
- Todo dentro de `DB::transaction`; numeración con `pg_advisory_xact_lock` (no-op en SQLite). Lanza `CheckoutException` ante cualquier fallo.

10.2 Pricing (Support\Pricing)

```
Pricing::bs(float $usd, float $rate, ?int $roundStep = null): float
# bs = usd * rate → ceil(bs / roundStep) * roundStep → round(bs, 2)
```

El redondeo `roundStep` (5, 10...) se usa en el POS para precios "redondos". Si `price_override` está activo, no se recalcula.

10.3 Créditos y cobro

- Una venta a crédito se crea con `payment_method='credito'`, `status='pendiente'`, `paid_at=null`; se genera un `CreditMovement` tipo `cargo` y se descuenta `customers.balance`.
- Cobro** (`CustomerController::payCredit`): crea `SalePayment(method=real, amount=deuda*tasa)` del día de cobro, `status='completada'`, `paid_at=now()` y sube el saldo. Los reportes suman **por fecha de cobro**.
- Anular crédito cobrado**: borra los movimientos `pago` (conserva el `cargo`), revierte `balance`, y la venta vuelve a `pendiente` (sin marcarla `anulada`).

10.4 Comandas

- Snapshot editable; el stock NO se descuenta al montar/entregar, solo al **cobrar** (reutiliza `CheckoutService` con `comanda_id`).
- Entrega por unidades: `delivered_quantity`; fila entregada cuando `delivered_quantity >= quantity`; la comanda pasa a `entregada` automáticamente.
- `order_type`: `local` | `para_llevar` | `delivery`. Solo `delivery` se cobra por adelantado.
- Numeración diaria `comanda_number` (0001...) calculada en `ComandaController::nextComandaNumber()`.
- No se puede eliminar una comanda `cobrada`.

10.5 Mermas y consumo interno (multilínea)

- La migración `2026_09_05_000003` añade `mermas.cost` (USD); el modal permite un carrito de N líneas, cada una con producto, cantidad y costo.
- Scopes en `Merma`: `mermaType()` (registros clásicos) y `consumption()` (tipo `consumo`). Reportes separan ambas poblaciones y muestran costo USD del consumo.
- El dashboard maneja un KPI "Consumo interno" con costo.

10.6 Numeración y transacciones

`sale_number` y `comanda_number` se generan dentro de la transacción con `pg_advisory_xact_lock` para evitar colisiones en accesos concurrentes; es no-op en SQLite para los tests.

11. Rutas de la aplicación

Todas las rutas viven en `routes/web.php`. Resumen por módulo (perfil del usuario mediante):

Módulo	Rutas (métodos / nombres)	Protección
Auth	<code>GET POST /login</code> , <code>POST /logout</code> , <code>POST /lanzador/cerrar-sesion</code>	público (login) / auth
Dashboard	<code>GET /dashboard</code>	auth
POS	<code>GET POST /pos</code> , <code>GET /pos/products</code>	auth
Comandas	<code>index</code> , <code>history</code> , <code>create</code> , <code>store</code> , <code>show</code> , <code>update</code> , <code>mark-delivered</code> , <code>deliver-item</code> , <code>collect</code> , <code>close</code> , <code>destroy</code> (todos <code>GET POST PUT PATCH DELETE</code>)	auth
Ventas	<code>index</code> , <code>show</code> , <code>destroy</code>	consultas auth; <code>destroy</code> con <code>can:void-sales</code>
Productos / Categorías / Combos	CRUD completo + <code>bulk-toggle</code> , <code>bulk-delete</code> , <code>toggle-active</code>	<code>can:manage-products</code>
Inventario / Producción	<code>index store</code> y <code>index store destroy</code>	<code>can:manage-inventory</code>
Mermas	<code>index</code> , <code>store</code>	<code>can:manage-waste</code>
Créditos	<code>index store show update</code> + <code>POST credits/{customer}/credits/{sale}/pay</code>	<code>can:manage-credits</code>
Tasa de cambio	<code>index</code> , <code>store</code>	<code>can:manage-exchange-rates</code>
Reportes	<code>index</code> , <code>sales</code> , <code>products</code> , <code>credits</code> , <code>top-days</code> , <code>waste</code> , <code>profit-margin</code> , <code>sales-by-schedule</code> , <code>slow-movers</code> , <code>weekly-performance</code> , <code>production-vs-sales</code> — todos con exportación <code>CSV</code>	<code>can:view-reports</code>
Usuarios / Roles	CRUD completo + <code>toggle-active</code>	<code>can:manage-users</code>

12. Interfaz de usuario

- **Layout:** `resources/views/layouts/app.blade.php` (sidebar + offcanvas móvil). El menú refleja permisos con `@can`; los accesos rápidos del dashboard también están protegidos.
- **POS:** `pos/index.blade.php` + `resources/js/pos.js` (aplicación Alpine `posApp()`): filtros por categoría, búsqueda, paginación, carrito, modal de checkout con vista de recibo y modal de éxito. Los productos llegan desde el endpoint `/pos/products`.
- **Tablas filtrables:** patrón de “dos vacíos” (servidor vs filtro) usado en productos, con `applyFilters()`; el encabezado permanece siempre visible.
- **Comandas:** pantalla de “mini-POS” en `comandas/create` y edición en `comandas/show` (entrega por unidad, modal de cobro).
- **Pricing card:** en create/edit de productos con recálculo bidireccional digno de mantener coherente con `Pricing` del servidor.
- **Modales de crédito:** `credits/show.blade.php` con opciones de pago (clases `payment-option` / `selected`) y botón “Cobrar todo”.

13. Pruebas y calidad

13.1 Suite de pruebas

Framework: **PHPUnit** vía `./vendor/bin/phpunit` (23 archivos bajo `tests/`). Configuración: `RefreshDatabase` + SQLite in-memory; no requiere servicios externos. Estado: **304 pruebas, 990 aserciones, todas en verde**.

```
./vendor/bin/phpunit                # todo
./vendor/bin/phpunit --filter=test_name  # una prueba
./vendor/bin/phpunit tests/Feature/ProductTest.php  # un archivo
```

13.2 Pruebas por módulo (cobertura destacada)

- **Auth:** login por username, throttle (5 intentos), usuarios inactivos (login + expulsión), remember me.
- **Checkout/POS:** precios recalculados desde BD, validación de stock, numeración serializada, créditos, tasa obligatoria.
- **Comandas:** estados (montada→entregada→cobrada), entrega por unidades, cobro→sale vinculada, no se elimina cobrada.
- **Mermas:** carrito multilínea con costo, validación de stock por línea, separación merma vs consumo.
- **Créditos:** cobro real con método, anulación que abre crédito "pendiente".
- **Roles:** gates, rol de sistema no eliminable, permisos por módulo.

13.3 Herramientas estáticas

```
./vendor/bin/pint app/... tests/...  # estilo (PSR-12)
./vendor/bin/phpstan analyse --memory-limit=1G  # análisis estático (nivel 5)
```

- El config `phpstan.neon` cubre `Services`, `Support`, `Exceptions` y modelos tipados con `@property`/`@property-read`; no cubre controllers ni modelos legacy, para no romper el baseline con ignores.
- Convención: usar `$x->prop ?? fallback` en vez de `$x?->prop ?? fallback` (nullsafe.neverNull en la superficie analizada).
- Las relaciones Eloquent están tipadas con PHPDoc generics (`@return HasMany<Item, $this>`, `@property-read Collection<int, Item>`).

14. Despliegue y producción

14.1 Escenario soportado

PC local con Windows + PostgreSQL + PHP CLI + Node. El **lanzador** (PowerShell) levanta `artisan serve`, purga sesiones y registra el token de sesión del lanzador. No se usa un servidor web tipo IIS/Apache para las peticiones.

14.2 Checklist de producción

1. Cambiar el password del usuario `admin` y crear cuentas individuales por persona.
2. Revisar `APP_URL`, `APP_ENV=production` y `APP_DEBUG=false`.
3. `composer install --no-dev --optimize-autoloader` y `npm run build`.
4. Cachear: `php artisan config:cache`, `route:cache`, `view:cache`.
5. Respalos de `casita` (ver §15).
6. Verificar red/periféricos: impresora de recibo impresa por el navegador (print CSS) si se requiriera.

Al tocar templates Blade: limpiar las vistas compiladas antes de exponer el cambio: `Remove-Item storage/framework/views/*.php -Force` (o `php artisan view:clear`).

15. Mantenimiento, respaldos y solución de problemas

15.1 Tareas de mantenimiento

Tarea	Comando
Limpiar sesiones expiradas / huérfanas del lanzador	<code>php artisan sessions:purge</code>
Limpiar vistas compiladas	<code>Remove-Item storage/framework/views/*.php -Force</code>
Recompilar assets tras cambios CSS/JS	<code>npm run build</code>
Actualizar catálogo de permisos/roles (idempotente)	<code>php artisan db:seed --force</code>

15.2 Respaldos (PostgreSQL)

```
& "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" -h 127.0.0.1 -U postgres -Fc casita -f casita.dump
# restaurar:
& "C:\Program Files\PostgreSQL\18\bin\pg_restore.exe" -h 127.0.0.1 -U postgres -d casita_restore casita.dump
```

También copiar `.env` y cualquier imagen subida (p. ej. las de combos, carpeta pública). Programar respaldo diario y verificar la restauración periódicamente.

15.3 Problemas comunes

Síntoma	Causa probable y solución
"No hay tasa de cambio registrada" en el POS	El checkout exige tasa vigente. Registrar una en el módulo Tasa de Cambio.
El checkout falla por stock	Productos <code>inventariable</code> sin existencias. Registrar entrada/producción.
Login bloqueado tras 5 intentos	Throttle temporal. Esperar el contador; de persistir, revisar <code>is active</code> del usuario.
Vista con plantilla vieja	Vistas compiladas: <code>Remove-Item storage/framework/views/*.php -Force</code> .
Assets sin estilos nuevos	Ejecutar <code>npm run build</code> (o <code>npm run dev</code> si se desarrolla).
DB "borrada" al clonar/crear base docs	Usar <code>DROP DATABASE IF EXISTS</code> solo sobre bases de prueba; respaldar la base real antes de operaciones destructivas.
Migración de <code>sale_payments.method</code> falla en SQLite	La migración usa <code>string()->change()</code> ; no intentar crear enums nativos manualmente.

16. Registro de cambios

Fecha	Cambio	Notas
2026-09	Consumo interno multilínea con costo (mermas)	Nueva columna <code>mermas.cost</code> , modal carrito, KPI en dashboard, reportes separan merma vs consumo.
2026-09	Reportes: separación de consumo interno	Reporte de mermas y CSV con columna Tipo/Costo USD; Producción vs Ventas con columna "Consumido".
2026-09-12	Tasas congeladas y créditos cobrados	<code>sales.rate</code> / <code>sales.paid_at</code> ; método <code>credito</code> en <code>sale_payments</code> ; ingresos por fecha de cobro.
2026-09-05	Combos finalizados y consumo	<code>combos.image</code> ; <code>round_bs</code> en productos y combos; <code>mermas.type</code> / <code>cost</code> .
2026-09-04	Comandas: tipos, notas y pagos	<code>comanda_items.order_type/note/collected</code> ; nueva tabla <code>comanda_payments</code> .
2026-08-31	Módulo de comandas	Tablas <code>comandas</code> / <code>comanda_items</code> , flujo montada→entregada→cobrada.
2026-08-24	Roles y permisos	Catálogo de 8 permisos, role/permission_role, gates dinámicos.

16.1 Correcciones de documentación detectadas

- README.md / AGENTS.md** mencionan `php artisan db:seed --class=DemoSeeder`; ese seeder **no existe** en el repositorio (solo `DatabaseSeeder` y `RolePermissionSeeder`). El dataset de demostración de este manual se construyó con un script temporal vía Tinker sobre la base `casita_docs`, sin tocar la base real.
- Los conteos de pruebas citados en documentación antigua (p. ej. "224+ tests") están desactualizados; la suite actual reporta **304 pruebas / 990 aserciones**.
- El proyecto actualmente no tiene inicializado git (no es repositorio).